# Cómo trabajar juntos en Walmart Computer Vision

Vicente Canio · Paulo Guerra · Vicente Vilches

Guía práctica del equipo | 20 de octubre de 2026

## 1. Una copia por persona y un punto de encuentro

El repositorio común es **https://github.com/vcanio/Capstone2026**. Cada uno trabaja en su propia copia del repositorio. Los cambios se comparten mediante GitHub. El flujo acordado es: **actualizar main → crear rama → editar → probar → commit → push → pull request → revisión → merge → pull de los demás**.

`main` es la versión compartida que debería funcionar. Una rama es un espacio para preparar una mejora. Un commit guarda un conjunto de cambios en tu computador; push lo sube a GitHub. Un pull request (PR) propone integrar una rama en main. Merge integra los cambios. Pull descarga e integra las novedades de la rama actual.

### Dónde guardar cada cosa

| Si cambias... | Trabaja en... | Cómo se comparte |
|---|---|---|
| Script experimental de bordes | Experimentos/Bordes/gondola_bordes.py | Rama y PR con el archivo en su ruta actual. |
| Cálculo de ocupación de Paulo | Herramientas/Modelos/ocupacion_paulo.py | Rama y PR; incluir pruebas pertinentes. |
| Panel y lógica del laboratorio | Herramientas/Modelos/ | Rama y PR. |
| Demo portátil | Herramientas/Demo_lite/ | Rama y PR del código; regenerar ZIP al publicar una demo. |
| Instalación y reproducción | Herramientas/Reproducibilidad/ | Rama y PR. |
| Manuales o acuerdos | Documentacion/ y README.md | Rama y PR. |
| Datos, etiquetas o ensayo nuevo | Datos/ o Experimentos/ con versión nueva | Metadatos pequeños por Git; binarios compartidos aparte. |
| Fotos, pesos .pt y ZIP | Paquetes compartidos aparte | Solo la demo portátil va en la Release pública. |

No crear copias como `script_final_paulo_v3.py` para reemplazar un script existente. Modificar el original en la rama permite ver exactamente qué cambió y conservar su historia. Para un experimento distinto, sí se crea una carpeta con nombre propio.

**Acuerdo sugerido:** quien desarrolla abre el PR; otro compañero revisa; quien mantiene la entrega confirma la versión y publica sus activos. Estos son turnos de trabajo, no permisos ya configurados en GitHub.

# 2. Primera vez en el computador de cada uno

El propietario debe añadir como colaboradores a las cuentas de Paulo y Vicente Vilches en GitHub: Settings → Collaborators. Cada persona usa su propia cuenta. No compartir contraseña ni token. Con acceso de escritura pueden subir ramas al mismo repositorio; sin él, deben solicitar acceso o trabajar con un fork y un PR.

Instalar Git, Python 3.11/3.12 y Docker con contenedores Linux. Elegir una carpeta de trabajo, abrir PowerShell ahí y ejecutar:

```
git clone https://github.com/vcanio/Capstone2026.git
cd Capstone2026
git remote -v
git status
```

Configurar la autoría dentro de esa copia. Sustituir los dos valores por los de la persona que está trabajando; el correo puede ser el noreply de su GitHub.

```
git config user.name "TU NOMBRE Y APELLIDO"
git config user.email "TU_CORREO_DE_GITHUB"
```

Estos valores identifican commits; no inician sesión. Al hacer push, Git puede pedir autenticación mediante Git Credential Manager o el navegador.

Preparar datos y modelos e iniciar en CPU:

```
python Herramientas/Reproducibilidad/preparar.py --assets D:/Transferencias/Walmart
& ./Herramientas/Modelos/iniciar_panel.ps1 -Cpu -Build
```

Abrir **http://localhost:8090**. Para desarrollar, recibir los paquetes de `.publicacion/paquetes_desarrollo/` del mantenedor y usar su carpeta local. La Release pública contiene solo la demo Lite, sus cuatro modelos y bordes:

```
python Herramientas/Reproducibilidad/preparar.py --assets D:/Transferencias/Walmart
```

La ubicación de la carpeta puede ser distinta en cada computador; las rutas internas del repositorio deben mantenerse. La demo Lite permite probar fotos, pero para modificar el proyecto se usa el clon del repositorio.

**Caso del mantenedor actual:** la copia preparada para el primer commit está en `C:/Proyectos/Walmart/.publicacion/repositorio`. Después de publicarla, usar ese checkout o un clon nuevo como copia de desarrollo. No alternar ediciones entre el laboratorio original y el clon sin trasladarlas explícitamente.

# 3. Ejemplo: Paulo mejora su script

Supongamos que Paulo cambia el cálculo de ocupación. Trabaja desde la raíz de su clon, donde se encuentran README.md, Herramientas y Datos. Antes de empezar:

```
git status
git switch main
git pull --ff-only origin main
git switch -c paulo/mejora-ocupacion
```

Continuar solo si status no muestra trabajo pendiente. Si lo hay, guardarlo con un commit en su propia rama antes de cambiar. Si pull falla, leer el mensaje y resolver la diferencia antes de seguir. El nombre de rama del ejemplo se crea una sola vez; para volver a ella usar `git switch paulo/mejora-ocupacion`.

Paulo edita **Herramientas/Modelos/ocupacion_paulo.py**, guarda el archivo y ejecuta las comprobaciones de la página 6. Luego revisa y guarda únicamente su cambio:

```
git diff -- Herramientas/Modelos/ocupacion_paulo.py
git add Herramientas/Modelos/ocupacion_paulo.py
git diff --cached
git commit -m "Corrige ocupacion de niveles con espacios laterales"
git push -u origin paulo/mejora-ocupacion
```

Si cambió pruebas o documentación relacionadas, añadir también esos archivos por su ruta antes del commit. Evitar `git add .` en el trabajo diario para no incluir archivos de otro cambio sin revisarlos.

### Qué hace en GitHub

1. Abrir el repositorio y pulsar **Compare & pull request**. Si no aparece, ir a Pull requests → New pull request.

2. Elegir **base: main** y **compare: paulo/mejora-ocupacion**.

3. Describir el problema, el comportamiento nuevo y las pruebas realizadas. Adjuntar una comparación si afecta resultados visuales.

4. Solicitar revisión a Vicente Canio o Vicente Vilches. Corregir observaciones en la misma rama: otro commit y `git push` actualizan el mismo PR.

5. Cuando la revisión y las comprobaciones estén correctas, usar **Merge pull request**. Conservar la opción predeterminada del repositorio.

Ejemplo de descripción: “Corrige el cálculo al haber un hueco lateral. Probado con las mismas imágenes de desarrollo y con la suite de modelos; adjunto antes y después. No he usado el conjunto test para ajustar el umbral”.

**Subir una rama no actualiza main.** Hasta que se integre el PR, un pull de main no incluirá la mejora de Paulo.

# 4. Cómo bajan los demás el cambio

Después de integrar el PR de Paulo, Vicente Canio y Vicente Vilches ejecutan, cada uno en su propio clon:

```
git status
git switch main
git pull --ff-only origin main
git log -5 --oneline
```

Con el árbol de trabajo limpio, estos comandos actualizan main y permiten ver los últimos commits. El archivo de Paulo aparecerá actualizado en la misma ruta. No hay que descargarlo a mano, pegar código ni reemplazar la carpeta completa. Reiniciar o reconstruir el panel según la página 6 para ejecutar el código nuevo.

## Si Vicente Vilches actualiza otra cosa

Por ejemplo, Vicente Vilches mejora una instrucción en README.md:

```
git switch main
git pull --ff-only origin main
git switch -c vilches/aclara-instalacion
# Editar y guardar README.md
git diff -- README.md
git add README.md
git commit -m "Aclara inicio CPU del laboratorio"
git push -u origin vilches/aclara-instalacion
```

Abre un PR a main y otro compañero lo revisa. Después del merge, Paulo y Vicente Canio repiten el bloque de actualización de arriba. Vicente Canio usa el mismo flujo para sus mejoras, con ramas como `canio/mejora-panel`.

## Revisar una rama antes del merge

Si quieren ejecutar el cambio de Paulo antes de aprobarlo, con el trabajo local guardado pueden descargar sus ramas:

```
git fetch origin
git switch --track origin/paulo/mejora-ocupacion
```

El segundo comando crea la rama local una vez. Si ya existe, usar `git switch paulo/mejora-ocupacion` y luego `git pull --ff-only` para actualizarla. Probar el cambio; después volver con `git switch main`. No hacer commits en la rama de otra persona salvo acuerdo explícito.

**Si tienes cambios sin guardar:** crea o vuelve a tu rama, revisa y haz un commit con los archivos correspondientes. No usar reset --hard, descartar archivos ni push --force para solucionar una actualización rutinaria.

# 5. Cuando dos personas trabajan al mismo tiempo

Antes de empezar, acordar por el canal habitual quién toca cada componente. Ejemplo: Paulo trabaja en ocupación, Vicente Canio en el panel y Vicente Vilches en documentación. Cada tarea tiene su rama y su PR. Los cambios relacionados pueden incluir varios archivos; la rama representa una mejora concreta.

Si tocan archivos o líneas diferentes, Git suele poder integrarlos. Si tocan las mismas líneas, puede pedir resolver un conflicto. El segundo PR debe incorporar el main actualizado antes de terminar su revisión.

## Actualizar tu rama con lo que ya integraron

Ejemplo para Paulo, después de guardar su trabajo con un commit:

```
git switch paulo/mejora-ocupacion
git status
git fetch origin
git merge origin/main
```

Si no hay conflictos, ejecutar pruebas y `git push`. El PR se actualiza. Si hay conflictos, Git indica los archivos afectados; `git status` los enumera.

1. Abrir cada archivo en el editor. Los bloques delimitados por `<<<<<<<`, `=======` y `>>>>>>>` muestran las dos versiones.

2. Revisar con la otra persona qué comportamiento debe quedar. Combinar las partes necesarias y borrar los marcadores; no elegir “todo mío” por rutina.

3. Guardar y comprobar que el resultado funciona.

4. Marcar cada archivo resuelto, completar el commit y subir:

```
git add RUTA_DEL_ARCHIVO_RESUELTO
git commit -m "Integra main y resuelve conflicto de ocupacion"
git push
```

`RUTA_DEL_ARCHIVO_RESUELTO` es un marcador: sustituirlo por la ruta que mostró Git. Repetir add por cada archivo resuelto. Si necesitan detener la resolución, pueden usar `git merge --abort` mientras ese merge siga pendiente; vuelve al estado anterior al intento. Haber hecho commit antes del merge protege el trabajo propio.

## Si pull rechaza la actualización de main

`--ff-only` se detiene cuando no puede avanzar sin combinar historias. Revisar `git status` y `git log -5 --oneline`; pedir apoyo al compañero que mantiene el repositorio para pasar los commits propios a una rama. No forzar la actualización.

Una mejora experimental puede seguir en su rama hasta estar lista. Hacer push permite compartirla y respaldar sus commits sin integrarla todavía en main.

# 6. Comprobar y ejecutar el código actualizado

Antes del PR, revisar `git diff` y ejecutar las comprobaciones pertinentes. Las pruebas se ejecutan con el Python del entorno de desarrollo que tenga las dependencias instaladas; el Python usado solo para descargar ZIP puede no tenerlas.

| Cambio | Comprobación desde la raíz del clon |
|---|---|
| Preparación o CVAT opcional | python -m unittest discover -s Herramientas/Reproducibilidad/tests -v |
| Demo Lite | python -m unittest discover -s Herramientas/Demo_lite -v |
| Datos y particiones | python -m unittest discover -s Herramientas/Datos -v |
| Panel, ocupación o modelos | python -m unittest discover -s Herramientas/Modelos -v |
| Antes de publicar código | python Herramientas/Reproducibilidad/verificar_publicacion.py |

El workflow de GitHub ejecuta una comprobación ligera en push y PR; no entrena modelos ni valida toda la calidad de las anotaciones. Revisar la pestaña Checks del PR y añadir qué pruebas locales pasaron. Un cambio de algoritmo necesita comparar resultados de desarrollo, además de pasar pruebas de código.

## Git pull y Docker son pasos distintos

Pull actualiza los archivos del computador. En el arranque normal, el código del panel está copiado dentro de la imagen Docker: hay que reconstruirla para aplicar los cambios. Esperar a que terminen entrenamientos o tareas activas y ejecutar:

```
& ./Herramientas/Modelos/iniciar_panel.ps1 -Cpu -Build
```

Volver a cargar el navegador. Si usan GPU o CVAT, conservar sus opciones de inicio. No reconstruir ni reiniciar el contenedor mientras otro proceso local esté entrenando o procesando una tarea que necesiten conservar.

Para desarrollo existe `-Dev`: monta Herramientas/Modelos desde el clon. En ese modo, panel.html se lee de nuevo al recargar; un cambio Python requiere reiniciar el servicio cuando esté libre. Cambios de dependencias o de otros directorios copiados en la imagen requieren Build. Usar el arranque normal de arriba como referencia común si no están seguros.

## Verificación manual común

Elegir V2-YOLO26 y la imagen `lider_express_040_0_original.jpg` del catálogo. Con confianza 0,25 y NMS 0,70, la referencia conservada tiene cinco niveles: cuatro suficientes y uno reponer. Si el cambio altera ese resultado, explicar por qué y revisar más casos; acertar una imagen no demuestra generalización.

# 7. Datos, modelos y CVAT: cómo compartirlos

**Código pequeño y documentación van por Git. Los paquetes grandes del laboratorio se comparten por separado. La Release pública contiene solo la demo con modelos y bordes. Las anotaciones se coordinan en el CVAT compartido.**

## Datos y anotaciones

Cada integrante entra en CVAT con su cuenta y guarda allí sus anotaciones. Coordinar quién anota y quién revisa cada tarea. Evitar editar simultáneamente la misma imagen. Git no combina dibujos que estén dentro de la base de CVAT. Una vez revisados, el responsable genera una exportación con versión nueva y conserva inventario, clases, partición y hashes.

No modificar los datos congelados de los cuatro ensayos de esta entrega. Cambiar una etiqueta exige otra versión. Mantener original y variantes de una misma familia en el destino previsto; conservar el test reservado para la evaluación final, sin usarlo para elegir umbrales o modelos.

## Entrenamientos nuevos

Usar una carpeta/identificador único, por ejemplo `paulo_yolo26_20261021_01`. Guardar configuración, versión de datos, entorno, métricas y conclusiones. El archivo best.pt y los resultados visuales grandes se comparten en paquetes del laboratorio; no añadir pesos al historial. No sobrescribir los cuatro ensayos conservados.

El script `Herramientas/Reproducibilidad/repetir_ensayo.py` permite repetir un ensayo congelado con otro nombre. Las opciones están en la guía de reproducción. Una ejecución con menos épocas sirve como prueba de funcionamiento, no como reproducción de las métricas del entrenamiento completo.

## Qué hacen los compañeros al recibir paquetes nuevos

1. Actualizan main con pull para recibir el manifiesto correspondiente.

2. Reciben los ZIP correspondientes, leen las instrucciones y ejecutan el preparador:

```
python Herramientas/Reproducibilidad/preparar.py --assets D:/Transferencias/Walmart
python Herramientas/Reproducibilidad/preparar.py --verificar
```

Si solo cambió código y el manifiesto sigue igual, no hace falta descargar de nuevo los datos. El preparador conserva archivos idénticos y rechaza sobrescribir uno diferente: guardar el trabajo propio y usar una copia limpia si hay colisión.

Cada persona configura sus accesos de CVAT localmente según la guía de réplica. No subir .env, contraseñas, acceso_local.json, entornos Python, fotos de pruebas privadas ni respaldos de CVAT. Tener permiso para colaborar en el código no implica permiso para publicar cualquier fotografía del cliente.

# 8. Publicar entregas y rutina del equipo

Integrar un PR de código actualiza main. **Eso no crea una Release ni actualiza automáticamente el ZIP portátil que ya descargó otra persona.** Si una mejora debe llegar al evaluador mediante Lite, regenerar y publicar otra entrega de la demo después de probarla.

Para la primera entrega, el mantenedor sigue Documentacion/Publicar_GitHub.md: commit y push del checkout preparado y Release con tag `avance-2026-10-20`. Solo adjunta Walmart_Lite_V2_Bordes_20261020.zip de `.publicacion/releases/`; los paquetes del laboratorio quedan en `.publicacion/paquetes_desarrollo/`. El PDF de esta guía y su fuente Markdown se incluyen en el código del repositorio.

Para entregas posteriores, definir un tag nuevo y activos coherentes con esa versión. Actualizar nombres, URLs y hashes del manifiesto antes de integrar el PR de publicación. No reemplazar un ZIP publicado bajo el mismo nombre/tag con contenido distinto. El empaquetador permite indicar `--tag`, pero los nombres de esta entrega están fijados por fecha: revisar y adaptar el empaquetado para la siguiente versión. Mantener la Release enfocada en probar la aplicación. El PR debe contener el manifiesto y la documentación final. Crear la Release apuntando al commit aprobado y probar su descarga desde un clon nuevo. Si los datos necesitan distribución privada, acordar el canal autorizado y usar `--assets`; no adjuntarlos a una Release pública.

## Rutina de cada sesión

| Momento | Qué hace cada persona |
|---|---|
| Al empezar | Revisar status, guardar pendientes, actualizar main, crear o retomar su rama. |
| Durante | Editar en su ruta, probar y hacer commits de cambios coherentes. |
| Al compartir | Push de su rama y PR con objetivo, pruebas y resultados. |
| Al revisar | Leer el diff, comprobar el caso y revisar Checks; solicitar cambios si faltan. |
| Al integrar | Merge a main tras revisión; avisar qué PR quedó integrado. |
| Al recibir | Pull de main y reconstrucción del panel cuando corresponda. |
| Al cerrar | Commit de lo que deba conservarse y push de la rama; anotar pendientes en el PR. |

## Acuerdos mínimos antes de empezar

- Usar ramas personales como paulo/tema, canio/tema y vilches/tema.
- Un compañero revisa el PR antes de merge. El propietario puede configurar protección de main para exigirlo; esta preparación no la activa.
- Coordinar cambios que afecten el mismo archivo, tarea CVAT o versión de datos.
- Documentar resultado y limitación del ensayo; conservar las versiones usadas.
- Compartir enlaces de PR, no archivos sueltos que reemplacen la copia ajena.

## Documentos relacionados

- Instalación, CVAT opcional y repetición de ensayos.
- Primera publicación del código y los activos.
- Alcance de la verificación de esta entrega.
- Fuente editable de esta guía.
- Versión PDF para compartir.